NOTE Technically, the `PageModel` and `@model` directive are optional. If you don't specify a `PageModel`, the framework executes an implicit page handler, as you saw in chapter 15, and renders the view directly. It's also possible to combine the .cshtml and .cshtml.cs files into a single .cshtml file. You can read more about this approach in *Razor Pages in Action*, by Mike Brind (Manning, 2022).

In addition to the `@page` and `@model` directives, the Razor view file contains the Razor template that is executed to generate the HTML response.

17.2.2 Introducing Razor templates

Razor view templates contain a mixture of HTML and C# code interspersed with one another. The HTML markup lets you easily describe exactly what should be sent to the browser, whereas the C# code can be used to dynamically change what is rendered. The following listing shows an example of Razor rendering a list of strings representing to-do items.

Listing 17.2 Razor template for rendering a list of strings

```
@page
@{                                                    ❶
    var tasks = new List<string>                      ❶
      { "Buy milk", "Buy eggs", "Buy bread" };        ❶
}                                                     ❶
<h1>Tasks to complete</h1>                            ❷
<ul>
@for(var i=0; i< tasks.Count; i++)                    ❸
{                                                     ❸
  var task = tasks[i];                                ❸
  <li>@i - @task</li>                                 ❸
}                                                     ❸
</ul>
```

❶ Arbitrary C# can be executed in a template. Variables remain in scope throughout the page.

❷ Standard HTML markup will be rendered to the output unchanged.